

CO TOTA

Agente Multi-LLM de Turismo en Colombia

Arquitectura · MCP · RAG · Multi-LLM · FastAPI

Universidad Adolfo Ibáñez

Simulaciones Basadas en Agentes

Programa de Master en Inteligencia Artificial

2025

01 · Caso de Negocio

Problema

El turismo en Colombia crece a tasas de 12% anual, generando una demanda masiva de información turística personalizada. Los viajeros necesitan respuestas precisas, actualizadas y contextualizadas sobre destinos, atracciones, rutas y condiciones. Las soluciones existentes presentan tres brechas críticas:

- Información estática desactualizada respecto a la oferta turística real
- Respuestas genéricas sin validación de calidad ni veracidad
- Ausencia de memoria contextual entre interacciones del usuario

Solución: Agente Tota

Tota es un sistema de agentes de inteligencia artificial que combina tres capacidades únicas en el mercado LATAM:

Capacidad	Descripción
Conversación inteligente	GPT-4o genera respuestas contextualizadas con personalidad de guía turístico experto
Validación automática	Claude Sonnet fiscaliza cada respuesta detectando errores, PII y alucinaciones
Conocimiento actualizado	RAG semántico + API Colombia en tiempo real garantizan información vigente
Memoria de sesión	Redis persiste el historial completo por usuario para conversaciones fluidas
API REST moderna	FastAPI + Uvicorn permiten integración con cualquier canal digital

Propuesta de Valor

Diferenciador clave

Tota es el único agente turístico con fiscalización automática multi-LLM: OpenAI genera y Claude verifica, garantizando respuestas de alta calidad con trazabilidad completa.

Mercado Objetivo

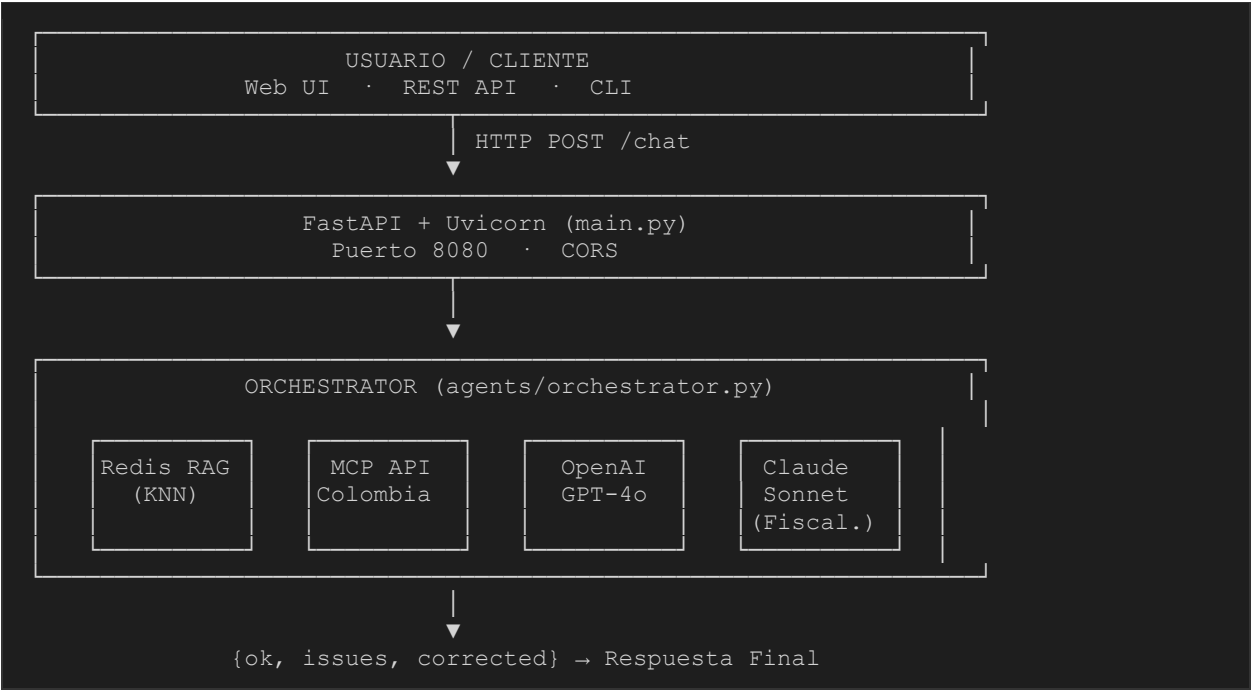
- Agencias de viaje digitales en Colombia y LATAM
- Ministerio de Comercio, Industria y Turismo de Colombia
- Plataformas de e-commerce turístico (Airbnb, Booking, Despegar)

- Aplicaciones móviles de guía turística
- Hoteles y operadores turísticos con chatbots de atención

02 · Arquitectura del Sistema

Diagrama de Alto Nivel

El sistema implementa un patrón de arquitectura de agentes con orquestación centralizada, donde cada componente tiene una responsabilidad única y bien definida:



Estructura de Archivos

```
colombia_tourism_agent/  
├── main.py                # FastAPI + CLI + UI web  
├── mcp_server_colombia.py # Servidor MCP (8 tools)  
├── .env                  # Variables de entorno  
├── requirements.txt  
├── agents/  
│   ├── orchestrator.py  # Pipeline principal  
│   ├── openai_agent.py  # GPT-4o conversador  
│   └── anthropic_fiscalizador.py # Claude validador  
├── services/  
│   ├── mcp_colombia_client.py # Cliente MCP + fallback  
│   └── redis_rag.py         # KNN semántico + sesiones
```

Flujo de Datos Detallado

Cada mensaje del usuario atraviesa un pipeline de 7 pasos antes de retornar la respuesta final:

Paso	Descripción
Paso 1	Cargar historial desde Redis (últimos 20 turnos de la sesión)
Paso 2	Búsqueda KNN semántica en Redis (top-5 atracciones relevantes)
Paso 3	Consulta en tiempo real a API Colombia vía MCP (atracciones, deptos)
Paso 4	GPT-4o genera respuesta con contexto RAG + API + historial
Paso 5	Claude Sonnet fiscaliza: verifica datos, detecta PII y alucinaciones
Paso 6	Si hay issues → Claude corrige; si está OK → respuesta original
Paso 7	Persiste turno en Redis y retorna ChatResponse al cliente

03 · Model Context Protocol (MCP)

¿Qué es MCP?

El Model Context Protocol (MCP) es un estándar abierto desarrollado por Anthropic en 2024 para estandarizar la comunicación entre modelos de lenguaje y fuentes de datos externas. Funciona como un 'USB para agentes de IA': define una interfaz universal que permite conectar cualquier LLM con cualquier servicio externo.

Analogía clave

MCP es para los agentes de IA lo que HTTP es para la web: un protocolo estándar que permite que cualquier cliente se comuniquen con cualquier servidor, independientemente de cómo estén implementados internamente.

Componentes del Protocolo

Componente	Descripción
MCP Server	Expone capabilities como Tools, Resources y Prompts. En este proyecto: <code>mcp_server_colombia.py</code>
MCP Client	Se conecta al server y llama tools. En este proyecto: <code>mcp_colombia_client.py</code>
Transport	Mecanismo de comunicación: stdio (subprocess) o SSE (HTTP streaming)
Tools	Funciones invocables por el LLM. Tienen nombre, descripción y schema JSON de parámetros
Resources	Datos que el LLM puede leer (archivos, URLs, DB). No usados en este proyecto
Prompts	Templates de prompts reutilizables. No usados en este proyecto

Implementación en Tota

El servidor MCP de Colombia expone 8 tools que abstraen completamente la API de `api-colombia.com`:

```
# mcp_server_colombia.py - Definición de un Tool MCP

from mcp.server import FastMCP

mcp = FastMCP(
    name="colombia-tourism",
```

```
        instructions="Servidor MCP de turismo en Colombia..."
    )

@mcp.tool()
async def search_attractions_by_city(city_name: str) -> str:
    """
    Busca atracciones turísticas filtrando por nombre de ciudad.
    Args:
        city_name: Nombre de la ciudad (ej: Cartagena, Medellin)
    Returns:
        JSON con lista de atracciones.
    """
    # ... lógica de consulta a api-colombia.com
    return json.dumps(results, ensure_ascii=False)
```

Transportes MCP: stdio vs SSE

Transporte	Descripción y cuándo usar
stdio (subprocess)	El client levanta el server como proceso hijo. Comunicación por stdin/stdout. Ideal para Claude Desktop y desarrollo en Linux/Mac.
SSE (HTTP)	El server corre independiente como proceso HTTP. El client se conecta via Server-Sent Events. Ideal para producción y Windows.
direct (fallback)	En Windows + uvicorn, el subprocess se bloquea. El fallback llama httpx directamente sin MCP. Mismo resultado, sin bloqueo.

Tools MCP Implementados

Tool	Descripción
get_touristic_attractions	Lista paginada de atracciones. Params: page, page_size
get_attraction_by_id	Detalle completo por ID numérico
search_attractions_by_city	Filtro semántico por nombre de ciudad
get_all_attractions_summary	Resumen completo (hasta 500 registros) para indexar en RAG
get_cities	Todas las ciudades de Colombia
get_departments	Todos los departamentos de Colombia
get_natural_areas	Áreas naturales protegidas y parques nacionales
get_airports	Aeropuertos con ubicación y tipo
build_realtime_context	Contexto enriquecido filtrado por query para los LLMs

04 · Arquitectura Multi-LLM

¿Por qué dos modelos?

La arquitectura multi-LLM resuelve un problema fundamental en sistemas de IA conversacional: ningún modelo es perfecto. Al combinar dos LLMs especializados con roles distintos, el sistema logra mayor calidad y confiabilidad que cualquier modelo único.

Roles y Responsabilidades

Atributo	OpenAI GPT-4o
Modelo	OpenAI GPT-4o
Rol	Agente Conversador (Tota)
Temperatura	0.7 (creativo)
Max tokens	1,200
Fortaleza	Generación fluida, conocimiento amplio, tono natural
Responsabilidad	Generar respuesta turística completa con contexto

El Agente Fiscalizador: Lógica de Validación

Claude recibe el trio {pregunta original, respuesta del agente, contexto de datos} y debe retornar un JSON estructurado con su evaluación:

```
# Respuesta OK – sin correcciones
{
  "ok": true,
  "issues": [],
  "corrected": null
}

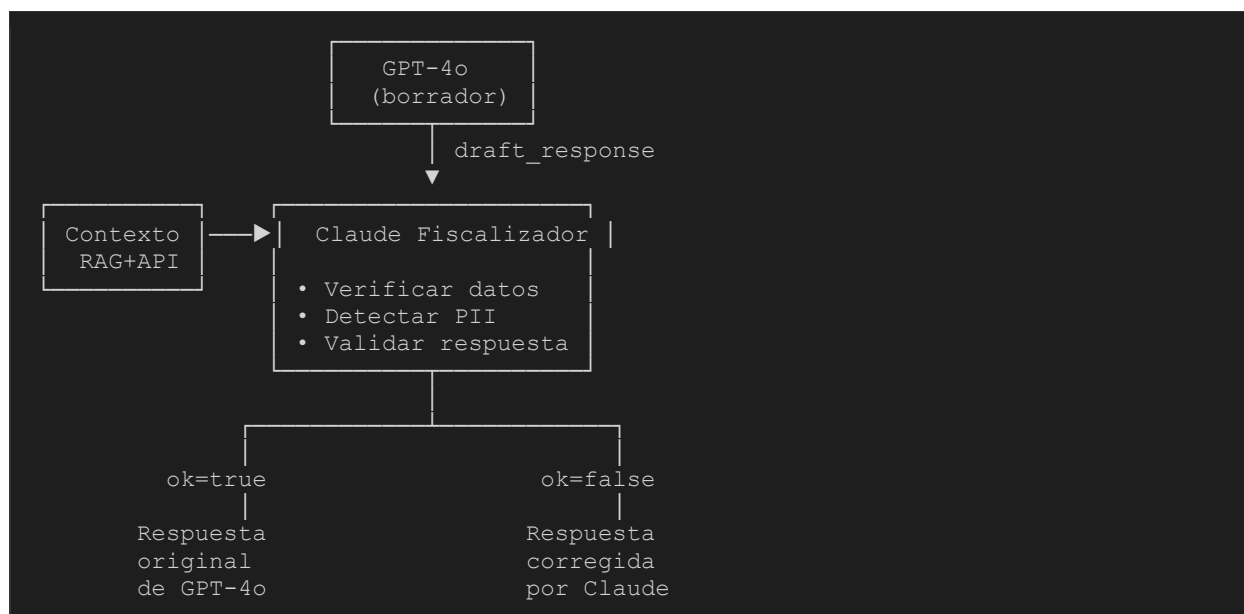
# Respuesta con problemas – Claude corrige
{
  "ok": false,
  "issues": [
    "El Museo del Oro no está en Medellín sino en Bogotá",
    "La Ciudad Perdida requiere 4 días mínimo, no 2"
  ],
  "corrected": "Versión corregida de la respuesta con datos precisos..."
}
```

Criterios de Fiscalización

- Verificar que atracciones y ciudades mencionadas coincidan con el contexto RAG

- Detectar datos geográficos incorrectos (ciudad/departamento equivocado)
- Identificar información potencialmente peligrosa para turistas sin advertencia
- Detectar datos sensibles o PII innecesarios en la respuesta
- Verificar que la respuesta responde efectivamente la pregunta original

Diagrama de Fiscalización



System Prompts

GPT-4o — Agente Tota

Prompt del agente conversador

Eres Tota, una experta asistente de turismo en Colombia con profundo conocimiento de cada rincón del país. Tu misión: orientar a los viajeros con información precisa, útil y entusiasta. Usa los datos de contexto (RAG + API en tiempo real) para dar respuestas concretas con nombres, descripciones y ubicaciones reales.

Claude Sonnet — Fiscalizador

Prompt del fiscalizador

Eres un agente Fiscalizador especializado en turismo colombiano. Tu rol es AUDITAR y VALIDAR respuestas de otro agente de IA. Detecta afirmaciones incorrectas o inventadas, verifica que los lugares coincidan con el contexto, identifica información peligrosa para turistas. Responde ÚNICAMENTE con JSON válido: `{ok, issues, corrected}`.

05 · Redis RAG — Búsqueda Semántica KNN

¿Qué es RAG?

Retrieval Augmented Generation (RAG) es una técnica que combina recuperación de información con generación de texto. En lugar de depender solo del conocimiento interno del LLM, el agente primero recupera información relevante de una base de conocimiento y luego la usa como contexto para generar la respuesta.

Pipeline de Indexación

```
Atracción turística (JSON)
    ↓
Construcción de texto
name + description + city + department + tags
    ↓
OpenAI text-embedding-3-small
(vector de 1536 dimensiones)
    ↓
Redis Storage
tourism:doc:<md5> → JSON metadata
tourism:emb:<md5> → bytes float32
tourism:index      → SET de doc_ids
```

Búsqueda KNN (k-Nearest Neighbors)

Cuando llega una consulta del usuario, el sistema realiza una búsqueda por similitud coseno sobre todos los embeddings indexados:

```
async def search(query: str, top_k: int = 5) -> list[dict]:
    # 1. Generar embedding de la query
    query_emb = await embed_text(query)

    # 2. Cargar todos los embeddings del índice (pipeline Redis)
    doc_ids = await r.smembers('tourism:index')

    # 3. Calcular similitud coseno con cada embedding
    scores = []
    for did in doc_ids:
        stored_emb = load_embedding(did)
        sim = cosine_similarity(query_emb, stored_emb)
        scores.append((sim, did))

    # 4. Retornar top-K documentos más similares
    return top_k_docs(scores)
```

Similitud Coseno

La similitud coseno mide el ángulo entre dos vectores en el espacio de embeddings. Un valor de 1.0 indica vectores idénticos (máxima similitud), 0.0 indica vectores ortogonales (sin relación):

```
similitud(A, B) = (A · B) / (||A|| × ||B||)

Ejemplo:
query: "playas caribe colombiano"
doc 1: "Ciudad amurallada Cartagena" → sim: 0.87  ✓ relevante
doc 2: "Cocora Valley Eje Cafetero"   → sim: 0.31  ✗ no relevante
doc 3: "Playa Blanca Bolívar"         → sim: 0.91  ✓ muy relevante
```

Memoria de Sesión

Redis también almacena el historial de conversación por sesión, permitiendo que el agente mantenga contexto entre turnos:

Atributo	Valor
Estructura de clave	chat:{session_id}:history
Tipo Redis	Lista (RPUSH para agregar, LRange para leer)
TTL	3600 segundos (1 hora de inactividad)
Formato	JSON por turno: {role: user assistant, content: string}
Límite	Últimos 20 turnos por consulta (configurable)

06 · Roadmap de Implementación

El proyecto implementa los 7 pasos del taller de la Guía de Agentes, siguiendo el roadmap para llevar un agente a producción:

PARTE 1 · Fundamentos y Arquitectura

Paso 01 — Documentos de Negocio

- Definición del dominio: turismo en Colombia como caso de negocio
- Fuente de datos: API Colombia (api-colombia.com) con 500+ atracciones
- Definición de queries objetivo: destinos, rutas, gastronomía, seguridad
- Formatos de respuesta estandarizados para el agente

Paso 02 — Embeddings → Vector DB (Redis)

- Modelo: text-embedding-3-small de OpenAI (1536 dimensiones)
- Chunking: cada atracción = 1 documento (name + description + city + tags)
- Vector DB: Redis con almacenamiento binario float32
- Metadata: id, nombre, ciudad, departamento, descripción, tags

Paso 03 — Agente Orquestador

- Clase TourismOrchestrator con pipeline de 7 pasos
- Herramientas: Redis RAG, MCP Colombia, OpenAI, Claude
- Criterios de delegación: RAG para conocimiento base, API para datos frescos
- Formato de salida: ChatResponse con metadata de fiscalización

Paso 04 — Agentes Workers

- Worker 1 — openai_agent.py: conversación con GPT-4o, system prompt turístico
- Worker 2 — anthropic_fiscalizador.py: validación con Claude Sonnet
- Worker 3 — mcp_colombia_client.py: consulta API en tiempo real
- Manejo de errores con tenacity (3 reintentos, backoff exponencial)

Paso 05 — Agente Fiscalizador

- Claude Sonnet valida cada respuesta antes de enviarla al usuario
- Detecta: datos incorrectos, PII, alucinaciones geográficas
- Retorna JSON estructurado: {ok, issues, corrected}
- Si ok=false y hay corrected → se usa la versión de Claude

Paso 06 — Búsqueda Semántica KNN

- k=5 documentos recuperados por consulta (configurable RAG_TOP_K)
- Similitud coseno sobre todos los embeddings del índice
- Métricas: similarity score por documento, visibles en la UI
- Re-ranking implícito: el LLM prioriza el contexto más relevante

PARTE 2 · Persistencia y Go-Live

Paso 07 — SQL para Registros (pendiente)

Para trazabilidad completa en producción, se recomienda implementar:

- Tabla interactions: id, session_id, timestamp, query, response, tokens, latency
- Conexión vía SQLAlchemy a PostgreSQL
- Registro de cada interacción del orquestador para auditoría
- Dashboard de métricas: latencia p50/p95, tokens por sesión, tasa de corrección

07 · API REST con FastAPI + Uvicorn

Endpoints

Endpoint	Descripción
GET /	Interfaz web de chat (HTML + JS embebido)
GET /health	Health check básico: {status: ok}
GET /status	Estado de Redis, API Colombia, modelos LLM
POST /chat	Enviar mensaje al agente y recibir respuesta
POST /index	Indexar atracciones en Redis (ejecutar una vez)
GET /history/{id}	Historial de conversación de una sesión
DELETE /history/{id}	Borrar historial de una sesión
GET /docs	Swagger UI autogenerado por FastAPI

Request / Response del endpoint /chat

```
# REQUEST - POST /chat
{
  "message": "¿Qué visitar en Cartagena?",
  "session_id": "abc12345",           // opcional, se genera si no se provee
  "fiscalize": true,                 // activar fiscalizador Claude
  "use_realtime": true               // usar API Colombia en tiempo real
}

# RESPONSE - 200 OK
{
  "session_id": "abc12345",
  "response": "Cartagena de Indias es una de las ciudades más...",
  "was_corrected": false,
  "fiscalization_ok": true,
  "issues": [],
  "rag_hits": 5,
  "realtime_available": true
}
```

Comandos de Ejecución

```
# Instalar dependencias
pip install -r requirements.txt

# Configurar variables de entorno
cp .env.example .env
# Editar .env con API keys y Redis Cloud credentials

# Indexar atracciones (una sola vez)
```

```
python main.py --index

# Modo API REST (recomendado para producción)
uvicorn main:app --host 0.0.0.0 --port 8080

# Modo chat interactivo CLI
python main.py

# Modo demo automático
python main.py --demo
```

Variables de Entorno

Variable	Descripción
OPENAI_API_KEY	API Key de OpenAI (GPT-4o + embeddings)
ANTHROPIC_API_KEY	API Key de Anthropic (Claude Sonnet fiscalizador)
REDIS_HOST	Host de Redis Cloud
REDIS_PORT	Puerto de Redis Cloud
REDIS_PASSWORD	Contraseña de Redis Cloud
OPENAI_MODEL	Modelo OpenAI (default: gpt-4o)
ANTHROPIC_MODEL	Modelo Claude (default: claude-sonnet-4-20250514)
MCP_TRANSPORT	Transporte MCP: direct (Windows) stdio sse
RAG_TOP_K	Documentos a recuperar en búsqueda KNN (default: 5)
EMBEDDING_MODEL	Modelo de embeddings (default: text-embedding-3-small)

08 · Stack Tecnológico

Librería	Uso en el proyecto
FastAPI 0.111+	Framework web async para la API REST
Uvicorn	Servidor ASGI de alto rendimiento para Python async
OpenAI SDK 1.30+	GPT-4o (conversación) + text-embedding-3-small (RAG)
Anthropic SDK 0.28+	Claude Sonnet 4 como agente fiscalizador
MCP SDK 1.0+	Protocolo estándar para conectar LLMs con servicios externos
Redis 5.0+ (Cloud)	Vector store para RAG KNN + historial de sesiones
httpx 0.27+	Cliente HTTP async para API Colombia
Pydantic v2	Validación de datos y schemas para FastAPI
tenacity	Reintentos con backoff exponencial para llamadas a APIs
python-dotenv	Gestión de variables de entorno
rich	Terminal output enriquecido para CLI
numpy	Operaciones vectoriales para similitud coseno

Decisiones de Diseño

- Async/await end-to-end: todo el pipeline es asíncrono para máximo throughput
- Pydantic v2 al nivel módulo: modelos fuera de funciones para evitar errores ForwardRef
- Fallback directo en Windows: MCP stdio se bloquea en Windows con uvicorn, el cliente detecta el OS y usa httpx directamente
- Redis Cloud: elimina la necesidad de infraestructura local, disponible globalmente
- Session ID por cliente: la memoria de sesión es responsabilidad del cliente (stateless API)

Consideraciones de Producción

Seguridad

En producción: agregar autenticación (API Key o JWT) al endpoint /chat, limitar rate por IP, y rotar las claves de OpenAI y Anthropic regularmente. Nunca exponer las API keys en el repositorio.

Escalabilidad

El sistema es stateless en la capa API (el estado vive en Redis Cloud). Se puede escalar horizontalmente con múltiples instancias de uvicorn detrás de un load balancer (nginx, AWS ALB). Redis Cloud maneja la concurrencia.

Observabilidad (Paso 07 pendiente)

Para producción completa: implementar logging de todas las interacciones en PostgreSQL con latencia, tokens usados, tasa de corrección del fiscalizador y satisfacción del usuario.

